



nextwork.org

Build a Security Monitoring System



Kehinde Abiuwa

Alarms (5)				
☐ Hide Auto Scaling alarms Clear selection Create composite alarm Actions ▾ Create alarm Search Alarm state: Any ▾ Alarm type: Any ▾ Actions status: Any ▾ < 1 >				
☐	Name ▾	State ▾	Last state update (UTC) ▾	Conditions ▾
☐	Secret is accessed	In alarm	2025-09-04 01:38:43	Secret is accessed >= 1 for 1 datapoints with minute



Introducing Today's Project!

In this project, I will demonstrate how to monitor and alert on secret access by wiring up CloudTrail (to record access events), CloudWatch Logs/metrics (to capture and filter those events), and SNS (to send notifications). I'm doing this project to learn how to build two alerting paths and compare them—so I can see which approach provides faster, clearer, and less noisy security alerts when secrets are accessed.

Tools and concepts

Services I used were AWS Secrets Manager, AWS CloudTrail, Amazon CloudWatch (Logs, metrics, alarms), Amazon SNS, Amazon S3, and the AWS CLI. Key concepts I learnt include securely storing/retrieving secrets, auditing API activity with CloudTrail, sending logs to CloudWatch Logs, building metric filters and alarms, triggering email alerts via SNS, using S3 for log storage, and wiring these pieces together for end-to-end monitoring and incident alerting.

Project reflection

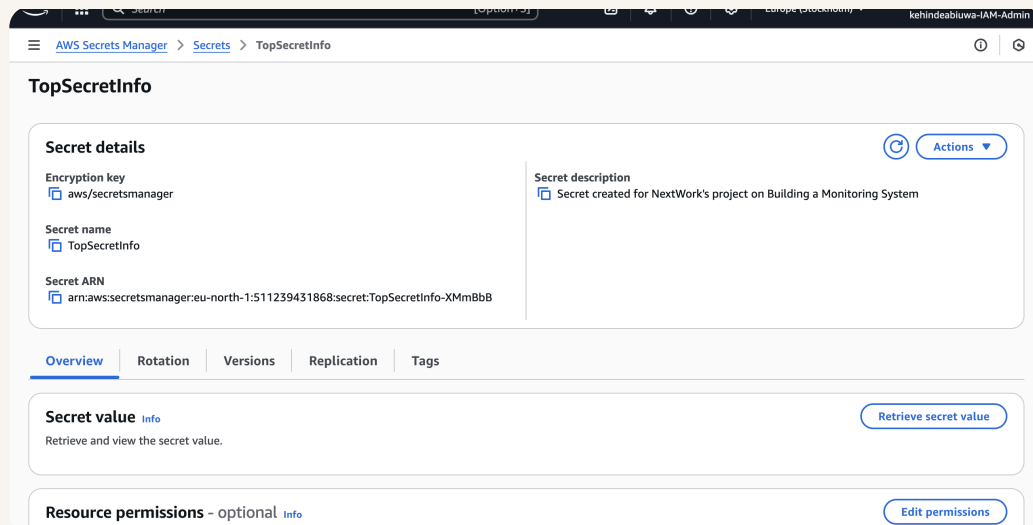
This project took me approximately 70 minutes. The most challenging part was troubleshooting why the CloudWatch alarm wasn't firing—tuning the metric filter/statistic to Sum, checking the log group, and making sure my SNS subscription was confirmed. It was most rewarding to see the whole pipeline work end-to-end: retrieve the secret, watch CloudTrail and CloudWatch record it, and then receive the SNS email alert confirming my monitoring is actually catching access.



Create a Secret

Secrets Manager is AWS's secure vault for sensitive data—like passwords, API keys, and other credentials—so you don't hardcode them in apps or share them in plain text. You could use Secrets Manager to store and fetch things like database passwords, third-party API tokens, and service credentials, with encryption and optional automatic rotation, and then retrieve them safely from your code when needed.

To set up for my project, I created a secret called TopSecretInfo that contains a single key/value pair: "The Secret is" → "rice is the best carb." It's stored in AWS Secrets Manager and encrypted with the default aws/secretsmanager KMS key





Set Up CloudTrail

CloudTrail is AWS's audit log—it records API activity across your account (who did what, when, from where, and against which resource), including user/role identity, source IP, request parameters, and results. I set up a trail to define what to record and where to store it: enable management events, have logs delivered to an S3 bucket, and keep them across regions. This lets me audit secret access later and trigger alerts when sensitive actions occur.

CloudTrail events include types like: Management events: These show you admin actions that configure your AWS resources - creating an EC2 instance, updating a security group, or in our case, accessing a secret. We're focusing on these in our project since secret access gets recorded here. Data events: These track high-volume actions that operate ON your resources rather than creating or configuring them - like uploading a file to S3 or running a Lambda function. Insights events: These detect unusual patterns in your management events, like someone suddenly creating 100x more IAM users than normal. Network activity events: These track network-related activities, like changes to your VPC configuration or traffic to a subnet.

Read vs Write Activity

Read API activity involves non-changing lookups, like ListBuckets, DescribeInstances, or viewing a secret's metadata. Write API activity involves state changes or sensitive data access, like Create, Update, Delete—and importantly, retrieving a secret's value (e.g., GetSecretValue) since it exposes data. For this project, we need to monitor Write activity, especially attempts to read the actual secret value, so we can alert on potential data exposure.



Verifying CloudTrail

I retrieved the secret in two ways: First through the Secrets Manager console, by opening my secret (TopSecretInfo) and clicking Retrieve secret value to view the key/value. Second using AWS CLI, by running: `aws secretsmanager get-secret-value --secret-id "TopSecretInfo" --region eu-north-1` in the terminal

To analyze my CloudTrail events, I visited CloudTrail → Event history and filtered for Secrets Manager. I found a GetSecretValue event from `secretsmanager.amazonaws.com` against my secret `TopSecretInfo`, showing who accessed it (my IAM user), when, from which source IP, the region (`eu-north-1`), the AWS access key used, and that the call was Read-only: true. This tells me CloudTrail is correctly recording secret reads, so I can trace who viewed a secret, what resource was touched, when it happened, and from where—exactly what I need to monitor sensitive access.

```
eu-north-1 console.aws.amazon.com/cloudshell/home?region=eu-north-1#
aws CloudShell
eu-north-1 +
~ $ aws secretsmanager get-secret-value --secret-id "TopSecretInfo" --region eu-north-1
{
  "ARN": "arn:aws:secretsmanager:eu-north-1:511239431868:secret:TopSecretInfo-XMmB8B",
  "Name": "TopSecretInfo",
  "VersionId": "35d3c9e7-d1b2-45d3-8471-b386a2619df7",
  "SecretString": "{\"The Secret is\":\"rice is the best carb\"}",
  "VersionStages": [
    "AWSCURRENT"
  ],
  "CreateDate": "2025-09-03T23:58:48.242000+00:00"
}
~ $
```



CloudWatch Metrics

CloudWatch Logs is a managed service that collects and stores logs from AWS services and your apps (including CloudTrail) in one place. It's important for monitoring because you can search those logs, create metric filters and alarms for patterns (like `GetSecretValue` on a secret), trigger notifications, visualize trends, and keep an auditable history for troubleshooting and security.

CloudTrail's Event History is useful for auditing who did what and when—an API activity ledger (last 90 days) and long-term, tamper-evident archives to S3 for compliance and forensics—while CloudWatch Logs are better for near-real-time monitoring and alerting: fast search, metric filters, alarms, dashboards, and notifications (e.g., trigger SNS when `GetSecretValue` appears).

A CloudWatch metric is a numeric time-series that you can alarm on or graph. When setting up a metric, the metric value represents the number that gets published when the log event matches your filter (e.g., publish 1 whenever a `GetSecretValue` appears). Default value is used when the log event doesn't match the filter (e.g., publish 0 so you still get a data point). Using 1/0 makes a clean binary signal you can sum, graph, or alarm on (e.g., "any secret access occurred").



aws [Search] [Option+S] Europe (Stockholm) Account ID: 5112-3943-1868 kehindeabiuwa-IAM-Admin

CloudWatch > Log groups > nextwork-secretsmanager-loggroup > Create metric filter

CloudWatch

Favorites and recents

Dashboards

► AI Operations [New](#)

► Alarms [1](#) [0](#) [0](#)

▼ Logs

Log groups

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

► Metrics

► Application Signals (APM)

► Network Monitoring

► Insights

Create new

Namespaces can be up to 255 characters long; all characters are valid except for colon(:) at the start of the name.

Metric name
Metric name identifies this metric, and must be unique within the namespace. [Learn more](#)

Secret is accessed

Metric name can be up to 255 characters long; all characters are valid except for colon(:), asterisk(*), dollar(\$), and space().

Metric value
Metric value is the value published to the metric name when a Filter Pattern match occurs.

1

Valid metric values are: floating point number (1, 99.9, etc.), numeric field identifiers (\$1, \$2, etc.), or named field identifiers (e.g. \$requestSize for delimited filter pattern or \$.status for JSON-based filter pattern - dollar (\$) or dollar dot (\$) followed by alphanumeric and/or underscore (_) characters).

Default value - optional
The default value is published to the metric when the pattern does not match. If you leave this blank, no value is published when there is no match. [Learn more](#)

0

Unit - optional
Select a unit

Cancel Previous Next

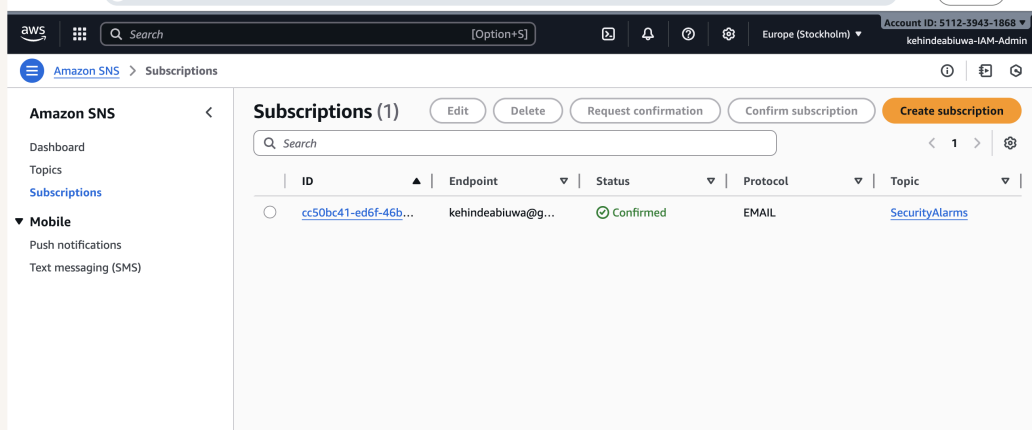


CloudWatch Alarm

A CloudWatch alarm is a rule that watches a metric and changes state (and can notify you) when that metric crosses a threshold. I set my CloudWatch alarm threshold to a static value of 1, with condition Greater/Equal (≥ 1), so the alarm will trigger when the `SecretsAccessed` metric emits a 1—i.e., whenever a `GetSecretValue` event appears in the logs (any secret access)

I created an SNS topic along the way. An SNS topic is a publish/subscribe channel in Amazon Simple Notification Service that lets services (like CloudWatch Alarms) send one message to many subscribers (email, SMS, HTTP, Lambda, etc.). My SNS topic `SecurityAlarms` is set up to notify my email whenever the CloudWatch alarm goes In alarm—i.e., when someone accesses the secret.

AWS requires email confirmation because SNS must verify that you actually own (and consent to use) the email address being subscribed. This helps prevent accidental or malicious subscriptions, cuts down on spam and bounces, and ensures alerts only go to endpoints that have explicitly agreed to receive them.





Troubleshooting Notification Errors

To test my monitoring system, I retrieved my secret again to generate a `GetSecretValue` event and expected CloudWatch → Alarm → SNS to send me an email. The results were that no email arrived. I'll troubleshoot by confirming my SNS subscription is confirmed, the metric filter matches `eventName = "GetSecretValue"`, the CloudTrail log group is receiving events in the same Region, the alarm moved to ALARM (threshold ≥ 1 , correct period), and the alarm's notification action points to my SNS topic (and isn't going to spam).

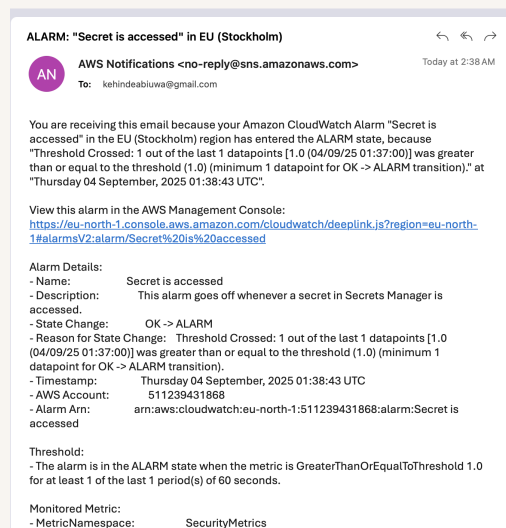
When troubleshooting the notification issues I: Checked CloudTrail Event history to confirm a `GetSecretValue` event for my secret actually exists (right user/region/time). Verified the trail is sending logs to CloudWatch Logs (CloudWatch log group is set on the trail, IAM role OK, new log streams/events are arriving). Reviewed the CloudWatch metric filter to be sure it matches the events (e.g., `eventSource = secretsmanager.amazonaws.com` and `eventName = GetSecretValue`, correct log group/namespace). Confirmed the CloudWatch Alarm is evaluating the metric (period/statistic set, threshold ≥ 1 , recent datapoints present) and that its alarm action points to my SNS topic. Made sure SNS can deliver: the topic exists in the same Region, my email subscription is Confirmed, and I checked spam/filters; then re-tested to see the email arrive.

I initially didn't receive an email before because CloudWatch's Alarm wasn't triggering an action. The metric was evaluated with Average over a 5-minute period, so a single `GetSecretValue` event averaged out and never crossed my ≥ 1 threshold. The key solution was to edit the alarm to use Statistic = Sum (count all events in the period) and shorten the Period to 1 minute, with the SNS topic set as the alarm action. After that change, re-accessing the secret triggered the alarm and the email.



Success!

To validate my monitoring setup, I triggered another GetSecretValue on my secret. I checked CloudWatch and saw the "Secret is accessed" alarm change from OK → ALARM (1/1 datapoint $\geq$ threshold 1). I received an AWS Notifications email from SNS confirming the alarm fired with a link to the alarm and the timestamp—proof that the logs, metric filter, alarm, and SNS alert all work end-to-end.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

